

Network Data Transfer Reference

BIOPAC Systems, Inc.

Network Data Transfer Reference

BIOPAC Systems, Inc.

Copyright © 2017-2018

Table of Contents

1. Introduction	1
2. Differences from the BIOPAC Hardware API	2
3. Network Data Transfer Overview	4
4. Data Connections	6
4.1. Variable Sampling Rates	6
4.2. Transfer Types	6
4.2.1. Single Connection	6
4.2.2. Multiple Connection	7
4.2.3. XML-RPC	8
4.2.4. Connections and Autosave File Acquisition Mode	8
4.3. Transport Protocol	8
4.3.1. TCP/IP	8
4.3.2. UDP	9
4.3.3. XML-RPC	10
4.4. Real-time Delivery Guarantees	10
4.5. Data Formats	11
4.6. Default Data Connection Settings	11
5. Locating AcqKnowledge Servers	13
6. Control Connections	14
6.1. TCP Port	14
6.2. Control Procedure Calls	14
6.2.1. Channel Index Parameter Structures	15
6.2.2. Querying Acquisition Parameters	15
6.2.3. Data Connection Configuration Commands	17
6.2.4. Reading Data During Acquisition	22
6.2.5. Other Control Connection Commands	22
7. AcqKnowledge Network Preferences	25

1

Introduction

Network data transfer is an AcqKnowledge feature that allows data during acquisitions to be transferred to external applications through TCP or UDP connections. All types of channels may be streamed during acquisitions, including calculation channels. This allows real time signal processing to be designed and performed within the regular AcqKnowledge environment while performing other further data processing and display within your own application.

Differences from the BIOPAC Hardware API

The BIOPAC Hardware API is a product that allows for direct integration of hardware communications with your own applications. It allows for direct configuration of MP devices and control over data acquisitions. The choice of whether to use the BIOPAC Hardware API or the network data transfer functionality of AcqKnowledge depends on the requirements of your project. Some of the major differences between the two are:

Functionality	BIOPAC Hardware API	AcqKnowledge Network Data Transfer
Configuring hardware	Direct access provided for sampling rate, triggering, and channels to be acquired.	Configuration must be contained within a graph template and cannot be directly modified.
Data delivery	Guaranteed. If interrupted, appropriate error codes are returned.	Not guaranteed. Client application must continually read data from data connections otherwise data may be silently dropped. The internal buffer within AcqKnowledge holds only two seconds of data.
Latency	Fast response times. Suitable for real-time applications.	Response time is not guaranteed and depends on multiple factors including calculation channel and network overhead. Suitable for slower moving signals or applications where accurate real-time data delivery is not critical.
Output control	Low-latency control over fixed analog output voltages and digital output.	Stimulators may be configured through graph templates to perform waveform output (MP150) or pulse generation starting with acquisition (MP36R). Provides control of constant analog output

Differences from the BIOPAC Hardware
API

Functionality	BIOPAC Hardware API	AcqKnowledge Network Data Transfer
		voltage and digital lines via XML-RPC, but latency is arbitrary and changes must generally be limited to 50 requests per second or less.
Calculation channels and signal processing	No signal processing or calculation algorithms are provided. You must implement signal processing algorithms within your own application code.	Calculation channel data may be streamed; all calculation channel types available within AcqKnowledge are available.
Programming requirements	DLL is accessible from a variety of languages; if no language binding is available, requires knowledge of how to map DLL calls to the chosen language.	Requires knowledge of network programming in chosen language, either TCP or UDP sockets. Requires an XML-RPC library for the chosen programming language and knowledge of how to use this library to write XML-RPC clients.
Supported MP hardware	MP150, MP36R. Versions for MP36 and MP35 are also available.	Only available for AcqKnowledge-supported MP device types (MP150, MP36R).
Allows for re-distributable applications	Yes, contact BIOPAC Systems, Inc. for licensing details.	No restrictions on your own application licensing or redistribution. However, users of your application will be required to purchase a full AcqKnowledge Development Edition license in order to use network data transfer infrastructure.
Multiple MP devices	No.	Requires multiple graphs and MP devices to be assigned to those graphs manually within the AcqKnowledge GUI, identical to the manual setup required for standalone AcqKnowledge multiple hardware operation. No automated support is available for configuring multiple hardware units.

It is not possible to use both the BIOPAC Hardware API and the network data transfer systems within the same application. Only one application may use the MP device at a time, so it is not possible to run both AcqKnowledge and an application using the BIOPAC Hardware API at the same time (or multiple applications using the BIOPAC Hardware API). It is best to consider the requirements of your application carefully before choosing which tool you use to interface with the data acquisition.

3

Network Data Transfer Overview

The network data transfer system consists of a pair of applications, AcqKnowledge and a custom application. The server is the AcqKnowledge process and the computer on which it is running. The client is the custom application that is to receive data from AcqKnowledge and the computer on which it is running. Note that the computer used to run the AcqKnowledge process and the custom application may be the same computer.

The real time data transfer system uses network connections between the server and the client to exchange information. There are two separate types of connections: data connections and the control connection. Data connections deliver binary data from AcqKnowledge to external applications during acquisitions. Data connections push data from the server to the client. Control connections are made from external applications to AcqKnowledge to query application state and adjust data connections. Control connections send configuration information from the client to the server.

All data connections are made using standard network protocols, either TCP or UDP. The choice of protocol is left up to the client. Single system image architectures should make connections using the loopback interface. If the server and the client are different computers, they must have appropriate IP networks in place that allow the computers to be identified either by IP address or by hostname. Firewalls must be properly configured to allow network communications between the client and server on the chosen ports for data and control connections. Appropriate network configuration is beyond the scope of this document and BIOPAC Systems, Inc.

The basic interaction between the client and the server during a particular session is as follows:

1. The AcqKnowledge application is launched, creating the AcqKnowledge server.
2. Client is launched and locates the AcqKnowledge server. This may be either through using a fixed or manually entered IP address in the client application, or through the client using the UDP broadcast mechanism for locating servers. See "Locating AcqKnowledge Servers" for more information on the broadcast approach.
3. Client uses a control connection to send appropriate configuration information to the server to configure the acquisition and provide instructions to the server on how data connections should be made. This may include transferring over a graph template if the template is not already open in AcqKnowledge.
4. Client creates appropriate sockets to listen for incoming data connections from the server.
5. Client uses a control connection to instruct the server to begin data acquisition.
6. Server makes data connections to the client and transfers binary data while acquisition is in progress. Client continually reads data off of the data connections and performs appropriate processing.

7. When acquisition completes, server closes data connections to the client.
8. Client uses a control connection to poll the server if data acquisition has completed.
9. Client performs required cleanup and exits.

4

Data Connections

Data connections are used to deliver data from the AcqKnowledge server to the client application. Data connections stream both data acquired from an MP unit and computed data from AcqKnowledge calculation channels. Data connections are available in a variety of formats and can be configured to meet the needs of the client application and bandwidth requirements.

Data connections are created only at the start of acquisitions. It is not possible to "attach" to data acquisitions that are already in progress. All data connections must be configured by clients prior to the start of data acquisitions.

XML-RPC calls for retrieving data are also described in this section. These connections are not persistent and not established from server to client, but rather a request made from client to server. The recommended mode of operation is regular TCP or UDP operation.

4.1. Variable Sampling Rates

AcqKnowledge uses variable sampling rates that allow different channels of data to be acquired at different sample rates. This allows fast moving signals (such as EMG) to be acquired at high sampling rates while simultaneously recording slow moving signals (such as respiration) at lower sampling rates. The primary use of variable sampling rate is to optimize storage requirements by minimizing space spent retaining slow moving data.

In addition to the storage optimizations, variable sampling rates are also used to optimize data connections. Channels that are sampled at lower rates require less network bandwidth to transfer from client to server. If channels of data cannot be disabled and overall bandwidth is limited, variable sampling rate can be used to lower the bandwidth required to move data between the server and the client.

The use of variable sampling rates requires the client to be aware of downsampled channels and handle data accordingly. Variable sampling rate may also affect the interpretation of the incoming data connection stream, as indicated below.

4.2. Transfer Types

Clients may choose between two different transfer types: single connection and multiple connection. As implied, single connection uses one data connection between the server and the client to deliver all data while multiple connection uses multiple data connections delivering data simultaneously.

4.2.1. Single Connection

The single connection transfer type uses a solitary connection between the server and the client to deliver data. When multiple channels of data are being delivered, all data will be sent over the single connection in an interleaved format. The interleaved format mingles the data of all channels in order. For example, an interleaved representation of two samples from three channels of data would be: C1 C2 C3 C1 C2 C3. This is equivalent to two sample frames where a frame contains the data at a particular sample location for all of the channels, in this case "C1 C2 C3". Frames are delivered sequentially over the data connection.

The single connection transfer type allows for all data to be processed from a single location and uses only one network connection resource. Single connection transfer has drawbacks including the need for the client to demux the data stream as well as account for variable sampling rates.

4.2.1.1. Variable Sample Rate Considerations

When variable sampling rate is being used, the single connection transfer type has the characteristic where not all frames have the same size. If a frame occurs at a sample position where a channel is not being sampled, that channel will not be included in the frame.

All downsampling within the AcqKnowledge variable sampling rate environment occurs at integer divisions of the base sampling rate. Only frames with indexes (zero-based) that are evenly divisible by the downsampling divider will contain data for that channel.

It is the responsibility of the client to compute and retain the frame index and to properly check channel dividers against the frame index to determine which channels are represented in that frame.

Frame 0 always contains a data point from every channel.

For example, consider a three channel acquisition. Channels 1 and 3 have a downsampling divider of 1, that is, they are being acquired at full speed. Channel 2 has a downsampling divider of 2, that is, channel 2 is being acquired at half speed. The first five frames of data in the single connection transfer type will appear as follows:

C1 C2 C3 | C1 C3 | C1 C2 C3 | C1 C3 | C1 C2 C3

Note how frames 1 and 3 are shorter as they occur at positions where channel two is not defined.

If variable sampling rate is being used, clients are required to perform proper frame indexing in order to demux the interleaved data stream. If clients cannot handle variable sampling rate correctly and are using the single stream transfer type, clients should check all of the downsampling dividers using the control connection to AcqKnowledge and warn the user if the configuration cannot be supported with that client.

4.2.2. Multiple Connection

The multiple connection transfer type uses a single connection from server to client for each channel of data that is being delivered. Using multiple connections offers the benefit of avoiding the client having the need to demux all of the data from each sample. If the client is operating in a high-load environment, the elimination of the demuxing may be useful in reducing processor overhead. It also allows client code to be simpler if variable sampling rate is being used.

The primary disadvantages of using the multiple connection transfer type are the usage of more network ports and the client assuming responsibility for synchronization of data across multiple channels. The data immediately available on one network connection may not be guaranteed to be the identical sample index of data being received for another channel. The client must keep track of sample index on an individual channel basis to properly synchronize data across multiple channels.

4.2.2.1. Variable Sampling Rate Considerations

Using variable sampling rate in a multiple connection transfer type is significantly easier. Each individual channel's connection delivers data at that channel's sampling rate. On a given connection, all samples have an identical length.

Clients still must be aware of variable sampling rates. The sampling rate of information on a downsampled channel connection will be different than the sampling rate of information on an upsampled channel connection. If the client is performing any time domain measurements or other computations involving the sample interval, the difference in inter-sample-interval must still be taken into account.

4.2.3. XML-RPC

The XML-RPC transfer type allows clients to explicitly request data from the server. Instead of data being automatically pushed to clients, the client must post an XML-RPC function call to the server over the control connection. This will allow the client to query the server for the most recently acquired data sample value for a particular channel. Clients that do not require continual data streams or interact with only very slow moving data may wish to use this method communication method to simplify implementation. This method returns values only; no information about sample indexes or lengths is returned.

XML-RPC has significant overhead for both client and server, so this transport method cannot handle more than a few requests per second. If faster response time is required, the client should implement either the single connection or multiple connection streaming methods.

XML-RPC is not a true data connection as it does not involve the server constructing a streaming connection to the client.

4.2.4. Connections and Autosave File Acquisition Mode

XML-RPC does not use data connections, instead reusing the single control connection. Data connections for binary data via TCP or UDP are created on the start of data acquisitions. When using graph templates configured to Autosave File acquisition modes a new graph file is created at the end of each acquisition, even in repeat mode. Unlike append to memory, when the graph file is saved at the end of an acquisition the data connections are terminated. When the new graph file is created for the subsequent acquisition, new data connections will be established.

The autorepeat interval should leave at least one second of downtime between subsequent repeats in order to allow the networking resources to be successfully closed and recreated for the new data segment. If insufficient time is left between acquisitions, networking connections may not be properly established. The repeat interval should be at least one second longer than the length of each individual acquisition to ensure proper operation.

4.3. Transport Protocol

Data connections offer a choice of using either TCP or UDP as the transport protocol for delivering data to the client. Choice of protocol depends on application requirements. When a client is receiving data, it is assumed that all data connections are using the same transport protocol.

4.3.1. TCP/IP

TCP is the preferred transport protocol. As TCP guarantees reliable, ordered delivery, all data is simply transferred from the server to the client without any additional information. Data will be streamed continuously as it becomes available. TCP is recommended for all clients that require a guarantee of receiving all information. It is also recommended for any configuration using up to two

computers. The port number used for data connections is specified by the client using the control connection prior to the start of acquisition. Once a client passes along port information, the client should begin listening for connections on that port.

When using TCP data connections, the start of acquisition is signaled to clients by the establishment of a connection on an appropriate port to the client. The end of an acquisition is signaled by the termination of the connection.

4.3.2. UDP

UDP is a connectionless protocol that does not guarantee either delivery or properly ordered reception of packets by clients. Data connections may be switched to UDP delivery mode. The primary benefit of using UDP datagrams is that a single data stream can be multicast to a number of computers. Multicasting is not offered implicitly by the AcqKnowledge data connection protocol but can be achieved implicitly by requesting a data connection be bound to a predetermined group broadcast address.

UDP delivery uses fixed size datagrams. The default datagram size is 512 bytes. Clients can modify this size to any fixed number of bytes prior to the start of acquisitions, but it is recommended the datagram size be evenly divisible by the size of the data type used for the channel or the entire size of a single sample frame for single connection transfers. The UDP packet size is stored in the template and is different for each graph. UDP clients that require a specific packet size should set that packet size prior to the start of each acquisition to ensure it is correct.

Each individual UDP datagram will have the following header and data contents:

Field	Data Type	Byte Offset	Description
Starting sample index	unsigned long (four bytes, network order)	0	This indicates the starting sample position of the first sample of data contained in the packet. This number is always monotonically increasing and can be used to reassemble datagrams that are received out of order. This field is always stored in network order. Usage of <code>htonl()/ntohl()</code> should convert from network order into host endian.
Length of data section in bytes	unsigned long (four bytes, network order)	4	Total number of bytes in the data section. Data is sent by AcqKnowledge as it is available, so not all data packets will contain the same amount of data. This field is always stored in network order. Usage of <code>htonl()/ntohl()</code> should convert from network order into host

Field	Data Type	Byte Offset	Description
			endian.
Data section	(type and endian specified by client)	8	Data section containing the binary data that was acquired. The data will be converted into the appropriate data as configured by the client.

All of the data delivery is performed as data is available from the MP unit. The actual size of the data section in packets that get delivered to the client is dependent on the speed of data acquisition and other activity on the host computer. If variable sampling rates are being used, individual sample frames will not be split up. All of the data for the final frame will be contained in the UDP packet. If a frame will not fit in the amount of space in a packet, it will be sent in the next packet.

There are no provisions for clients to request retransmissions of packets from the server. Clients should be aware that data may not be delivered when using UDP and should be prepared to examine sample indexes at the beginning of each packet and handle missing data accordingly (padding, displaying a warning to the user, etc.).

The port on which datagrams are delivered is specified by the client prior to the beginning of acquisition on the control connection. Once the client specifies its port, it should begin listening for datagrams delivered to that port. The start of acquisition will be signaled by the first datagram that is sent to that port. The sample index of the first datagram will correspond to the first hardware sample of data acquired by AcqKnowledge. If the graph is initially empty, the index will be zero.

Unlike TCP connections which get explicitly disconnected at the end of acquisitions, there is no direct messaging for UDP delivery indicating the end of acquisitions. Datagrams will not be sent to the client after the end of acquisition. Clients requiring explicit termination notification should either use TCP or implement a timeout mechanism combined with an XML-RPC `getAcquisitionInProgress` call on the control connection to locate the end of acquisitions.

4.3.3. XML-RPC

The XML-RPC "get most recent data sample" call is available on the control connection and uses standard XML-RPC connection semantics. Handling of XML-RPC can be performed by a library in the client's appropriate implementation language.

4.4. Real-time Delivery Guarantees

No delivery time guarantees exist for any data connection. Data is delivered to the client as it becomes available. The overall latency of the system from physical sample time to data delivery is dependent on a number of factors including network load, AcqKnowledge overhead (which is variable due to calculation channel processing time, user interface activity, thread scheduling, and other operations), system load, and other factors. The actual sampled data itself is guaranteed to be accurate; the sample time of a signal acquired into the MP unit is always accurate. It is only the time between the physical time corresponding to a sample and its delivery to the client application that is variable.

The AcqKnowledge server maintains an internal two second long data buffer. The client should continuously read data in order to create empty space in this buffer. If clients do not read data for over two seconds, the most recent data read from the MP unit will be dropped. TCP data connections provide no indicator of whether data was dropped; the sample index of consecutive UDP datagrams may be examined to determine if data has been dropped.

Clients that require strict real time guarantees or more predictable latencies should use the BIOPAC Hardware API.

4.5. Data Formats

It is assumed that the data transfer feature will be used in a mixed host environment, potentially with clients running in languages and environments that have restricted data types. The sampled information delivered by a data connection may be changed into a variety different formats for the client:

1. 64 bit floating point - This format is always available for all channels and may be delivered in either big endian byte order or little endian byte order. This is equivalent to a C style double data type.
2. 32 bit floating point - This format is always be available for all channels and may be delivered in either big endian byte order or little endian byte order. This data type is not native to AcqKnowledge, so the precision of received data may differ from data as recorded by AcqKnowledge. This is equivalent to a C style float data type.
3. Signed 16 bit integer - This format is available only for analog data channels. It may be delivered in either big endian byte order or little endian byte order. This is equivalent to a C style short data type. Clients receiving data in 16 bit integer format should use the control connection to determine appropriate floating point scaling factors that need to be applied to the data to convert into actual units.

Data formats should be specified by the client prior to the start of acquisition. If left unspecified, the default data format depends on the channel type:

Channel Type	Default Data Type
Analog	16 bit signed integer, little endian
Digital	16 bit signed integer, little endian
Calculation	64 bit floating point, little endian

XML-RPC get most recent sample requests will always return the double type of XML-RPC (in ASCII notation). It is not possible to get integer values via XML-RPC data requests.

4.6. Default Data Connection Settings

If the client does not modify any data connection settings, the following will be used:

1. Single connection transfer type
2. TCP/IP transport protocol
3. Port 15020 for single connection
4. For multiple connections, each channel type (16 channels per type) will be set as follows: analog channels [15020-15035], digit-

al channels [15040-15055], calculation channels [15060-15075].

5. Default data formats for each channel type as indicated in the "Data Formats" section

Note that no channels are enabled for delivery by default, so no data connections will be made at the start of acquisitions. Both data connection delivery and get most recent sample tracking are disabled. Even if the default settings are used, clients must still use a control connection to enable specific channels for data delivery through any of the methods.

5

Locating AcqKnowledge Servers

It is possible that clients and servers may be located on networks with dynamic IP addresses or other feature that make establishing the connection between machines difficult. In this case, a simple UDP broadcast mechanism can be used to locate known servers.

AcqKnowledge listens for incoming UDP packets on port 15012. If the UDP packet contains the sequence of ASCII characters "AcqP Client" and only that sequence, it will send a broadcast packet containing "AcqP Server Port:" followed by the port number on which the server is listening for control connections. The port number will be expressed in base 10 ASCII notation.

It is possible to configure AcqKnowledge to not acknowledge discovery requests for security purposes. See the "AcqKnowledge Network Preferences" section for more details.

6

Control Connections

Clients configure an AcqKnowledge server using control connections. A control connection allows the client process to control how data is going to be delivered to it, query settings, modify settings, and perform other basic operations without requiring direct user interaction with the graphical user interface of the AcqKnowledge application.

On the start of an acquisition, all data connections are established to the client that most recently established a control connection to the server. If more than two computers are used, however, the client may set a specific destination by using the `changeDataConnectionHostname` command.

It is possible to configure AcqKnowledge to not respond to any control connections for security purposes. See the "AcqKnowledge Network Preferences" section for more details.

The majority of all control connections use XML-RPC. The XML-RPC specification can be located at <http://www.xmlrpc.com/spec>. XML-RPC consists of an HTTP POST request being assembled by the client along with data content expressed in the XML-RPC notation. The remote procedure call will then return with an appropriate response to the caller. By design, XML-RPC is client and platform agnostic.

XML-RPC implementations exist for a wide variety of languages. If there is no implementation in the client's language, hardcoded requests can be embedded or a small helper application or shared library can assist in providing the control connection interface.

The URI that should be used when connecting to the server is the recommended `/RPC2` for XML-RPC. For example, a URL for localhost control connections to an AcqKnowledge server would be `http://localhost/RPC2`.

6.1. TCP Port

Only one control connection may be opened by a client to a server at a time. The server listens on port 15010 by default. As this port may conflict with other network services or may need to be modified for firewall accessibility, AcqKnowledge allows the server port to be modified by the user in the `Display > Preferences` panel. See the "AcqKnowledge Network Preferences" section for more details.

Clients should be aware that control connection port numbers are not fixed and should either allow users to change the port number from the client or use the dynamic discovery mechanism for locating AcqKnowledge servers.

6.2. Control Procedure Calls

Control procedure calls are remote procedure calls with a set method name and response. All method names and strings are case sensitive. The available control calls are roughly split up into querying acquisition parameters, configuring data delivery, and limited calls for modifying acquisition parameters and affecting application state.

6.2.1. Channel Index Parameter Structures

Some procedure calls take a channel index structure as a parameter. This is an XML-RPC structure that consists of the channel type and index. The channel type is a string member named "type" that is one of the following strings: analog, digital, calc. The channel index is an integer member named "index" and contains a zero-based index indicating the channel. For example, the following channel index parameter structure can be used to refer to calc channel 2 (recall, in AcqKnowledge calc channels are indexed from zero):

```
<struct>
<member>
<name>type</name>
<value><string>calc</string></value>
</member>
<member>
<name>index</name>
<value><int>2</int></value>
</member>
</struct>
```

6.2.2. Querying Acquisition Parameters

The calls for querying acquisition parameters are intended to allow clients to request information required for them to fill out appropriate parameters and to verify that previous control requests have been properly applied. The following control calls are recognized:

6.2.2.1. getMPUnitType

Method name: acq.getMPUnitType

Parameters: None

Return value: int

Retrieves the type of MP unit to which the server is connected. This may be zero (indicating no unit is connected, e.g. "No Hardware" mode) or one of the following: 100, 150, 35, 45.

The client can use this value to decide between appropriate templates to download or other channel settings.

6.2.2.2. getEnabledChannels

Method name: acq.getEnabledChannels

Parameters: string

Return value: array populated with int

Retrieves the channels that are available for acquisition and data delivery over a data connection to the client. The type of channels

that are to be returned are specified in the string parameter. The string parameter may be one of the following: analog, digital, calc. The type must be lowercase. "analog" returns information about analog channels, "digital" digital channels, "calc" calculation channels.

The enabled channels are returned as an array of channel indexes, zero-based. These are the indexes that can be validly used in calls for configuring data connection parameters.

6.2.2.3. getChannelScaling

Method name: `acq.getChannelScaling`

Parameters: channel index parameter structure

Return value: struct containing scaling

For channels that can be delivered in short integer format, the channel scaling provides the scale factor and offset used to convert the short into physical units using the formula: $\text{sample} * \text{scale} + \text{offset}$. The scaling is returned as a structure containing two double-valued members: "scale" contains the multiplication factor, "offset" contains the offset factor.

Channels that cannot be delivered as short integers cannot be scaled. If this method is called with an invalid type of channel, a fault response will be returned.

6.2.2.4. getChannelLabel

Method name: `acq.getChannelLabel`

Parameters: channel index parameter structure

Return value: string with channel label, or empty string if not applicable

Provides the user-visible label of the channel. This is the initial label assigned for the channel on the very first data acquisition.

6.2.2.5. getSamplingRate

Method name: `acq.getSamplingRate`

Parameters: None

Return value: double

Returns the current sampling rate expressed in Hz. This is the rate at which data is sampled for a channel whose downsampling divider is 1.

6.2.2.6. getDownsamplingDivider

Method name: `acq.getDownsamplingDivider`

Parameters: channel index parameter structure

Return value: int

Retrieves the downsampling divider for a particular channel. A channel is sampled at the base sampling rate divided by this factor.

This factor is also used to determine which frames contain samples for this channel.

6.2.3. Data Connection Configuration Commands

While it is possible to configure data connection parameters using AcqKnowledge, frequently clients may need to alter data connection configurations based upon dynamic information regarding the machine on which they are running (e.g. port collisions with other services, unexpected endian changes, etc.). The following commands may be used to configure how data is delivered to the application.

6.2.3.1. `getDataConnectionMethod`

Method name: `acq.getDataConnectionMethod`

Parameters: none

Return value: string

Returns the method currently being used for data connections between the server and client. The string value is either: single, multiple. "single" corresponds to a single data connection being made between the server and the client with all data interleaved over that connection. "multiple" corresponds to opening an individual connection to the client for each channel.

6.2.3.2. `changeDataConnectionMethod`

Method name: `acq.changeDataConnectionMethod`

Parameters: string

Return value: 0 on success, or fault code

Changes the method used to deliver data to the client. The parameter is a string that is one of the following: single, multiple. "single" opens up a single data connection and sends data in an interleaved fashion. "multiple" opens up an individual data connection for each channel.

6.2.3.3. `getDataConnectionHostname`

Method name: `acq.getDataConnectionHostname`

Parameters: none

Return value: string

Returns the currently set hostname for data connections. When data connections are established on the start of acquisition, the IP address associated with this hostname will be sent the data. Hostnames must properly resolve in order for data connections to be made; DNS configuration is outside of the scope of the network data transfer feature.

If this hostname is empty, data connections will be made to the most recently connected client application. This is the default setting.

Hostnames are unique on a graph by graph basis and are saved in templates and graph files. Clients redirecting data to another machine should check and set the data connection hostname as appropriate prior to the start of acquisition.

6.2.3.4. changeDataConnectionHostname

Method name: `acq.changeDataConnectionHostname`

Parameters: string

Return value: 0 on success, fault on error

Changes the destination machine for data connections, the recipient of the network delivered data. If the parameter is an empty string, data connections will automatically be made to the most recently connected client. The empty setting is the default.

If non-numeric hostnames are specified as parameters, they must be resolvable. If a hostname cannot be resolved, a fault will be returned. Proper DNS configuration of the server is beyond the scope of the network data transfer feature.

6.2.3.5. getTransportType

Method name: `acq.getTransportType`

Parameters: None

Return value: string

Retrieves the transport type that is being used to deliver data from the server to the client. The transport type is a string that is one of the following: `tcp`, `udp`. Note that the XML-RPC data delivery method may be used in addition to this transport type if channels are enabled.

6.2.3.6. changeTransportType

Method name: `acq.changeTransportType`

Parameters: string

Return value: 0 if successful, else fault code

Change the transport type that is used to deliver data from the server to the client. The transport type is a string that has one of the following values: `tcp`, `udp`. XML-RPC last value data delivery may be used in addition to this type provided channels are enabled properly.

6.2.3.7. getUDPPacketSize

Method name: `acq.getUDPPacketSize`

Parameters: None

Return value: int

Returns the current size in bytes of UDP packets that will be delivered to clients. Datagrams are always this fixed byte length although each individual datagram may contain varying amounts of data.

6.2.3.8. setUDPPacketSize

Method name: `acq.setUDPPacketSize`

Parameters: `int`

Return value: 0 on success, fault on error

Changes the size in bytes of UDP packets that are delivered to clients. Each individual datagram will always be this fixed length although the amount of data sent in specific packets may vary.

6.2.3.9. getUDPBroadcastEnabled

Method name: `acq.getUDPBroadcastEnabled`

Parameters: `None`

Return value: `boolean`

Determine if UDP packets are sent only to the client or are broadcast to the broadcast IP of the network. Broadcasting is supported only when the transport type is UDP.

6.2.3.10. changeUDPBroadcastEnabled

Method name: `acq.changeUDPBroadcastEnabled`

Parameters: `boolean`

Return value: 0 if successful, fault code on error

Modify whether UDP packets are sent only to the client or are broadcast to the broadcast IP of the network. Broadcasting is supported only when the transport type is UDP.

6.2.3.11. getSingleConnectionModePort

Method name: `acq.getSingleConnectionModePort`

Parameters: `None`

Return value: `integer`

Returns the port number on which the server will connect to the client to deliver data. This port is used only when the connection mode is set to "single" which interleaves all data over a single connection.

6.2.3.12. changeSingleConnectionModePort

Method name: `acq.changeSingleConnectionModePort`

Parameters: `integer`

Return value: 0 on success, else fault code

Modifies the port on which the server connects to the client to deliver data. This port is used only when the connection mode is set to "single" which interleaves all data over a single connection.

6.2.3.13. getDataDeliveryEnabled

Method name: `acq.getDataDeliveryEnabled`

Parameters: channel index parameter structure

Return value: boolean

Query whether a channel is enabled for data delivery. Channels must be enabled for data delivery in order for their data to be delivered to the client. Not all channels that are being acquired are required to be delivered to data delivery.

6.2.3.14. changeDataDeliveryEnabled

Method name: `acq.changeDataDeliveryEnabled`

Parameters: channel index parameter structure, boolean

Return value: 0 for success, else fault code

Change whether or not data delivery is enabled for a particular channel. Data delivery can only be changed prior to the start of an acquisition. Changes to data delivery enabling are only applied on the next start of acquisition.

6.2.3.15. getMostRecentSampleValueDeliveryEnabled

Method name: `acq.getMostRecentSampleValueDeliveryEnabled`

Parameters: channel index parameter structure

Return value: boolean

Query whether a channel is enabled for most recent data sample requests. If a client wishes to use the XML-RPC calls to fetch the most recent value of data acquired of a channel, the channel must be enabled for this functionality prior to the start of acquisition.

6.2.3.16. changeMostRecentSampleValueDeliveryEnabled

Method name: `acq.changeMostRecentSampleValueDeliveryEnabled`

Parameters: channel index parameter structure, boolean

Return value: 0 for success, else fault code

Change whether or not a channel is enabled for most recent data sample requests. When a channel is enabled, XML-RPC calls can be used during an acquisition to return the most recent sample of data acquired (or computed) for the channel. Any changes to the enabled state of a channel will be applied on the start of the next acquisition. If a client wishes to use XML-RPC calls to read the value of a channel, that channel must be enabled prior to the start of the acquisition.

6.2.3.17. getDataConnectionPort

Method name: `acq.getDataConnectionPort`

Parameters: channel index parameter structure

Return value: int

Retrieves the port on which the the server will deliver the data for the channel specified in the parameters to the client. Per-channel data connections are only used if the data connection method is set to "Multiple".

6.2.3.18. changeDataConnectionPort

Method name: `acq.changeDataConnectionPort`

Parameters: channel index parameter structure, integer

Return value: 0 for success, else fault code

Changes the port on which the individual connection is made by the server to the client to deliver the data for the channel specified in the parameters. This style of connection is used only if the data connection method is set to "Multiple".

6.2.3.19. getDataType

Method name: `acq.getDataType`

Parameters: channel index parameter structure

Return value: structure

Returns the data type that is being used in the binary data streams for the channel's data. The return value is a structure with a type and endian member. The type member, named "type", is a string and contains one of the following values: short, double, float. These strings correspond to their matching C-style data types. The endian member, named "endian", is a string and contains one of the following values: little, big. "little" corresponds to little endian byte order, big endian bit order. "big" corresponds to big endian byte order, big endian bit order.

6.2.3.20. changeDataType

Method name: `acq.changeDataType`

Parameters: channel index parameter structure, type structure

Return value: 0 on success, or fault code

Changes the data type that is used for binary data streams of the channel's data. The type structure is a struct containing two members. The "type" member is one of the following strings: double, float, short. Each string corresponds to the matching C-style data type. The "endian" member is one of the following strings: little, big. Each corresponds to the matching byte endian. Bit order within a byte will always be big-endian.

Not all channels may be able to support all data types. If the channel cannot be transmitted in the requested data type, a fault code will be returned.

6.2.3.21. setDataConnectionTimeoutSec

Method Name: `acq.setDataConnectionTimeoutSec`

Parameters: integer of new timeout in seconds

Return value: 0 on success, or fault code

Changes the timeout in seconds after which the data connections will be closed when acquisitions complete. Clients may need to set this keep-alive timeout in order to receive the trailing data at the end of acquisitions. The default value is "0" seconds which terminates all data delivery connections immediately when the final sample of data is acquired.

6.2.4. Reading Data During Acquisition

Normally data is delivered to clients during acquisitions using data connections. Data connections are either TCP or UDP connections established from the server to the client over which the server streams the incoming data. In some languages and environments however, it may not be possible to handle continuous data streams. The following XML-RPC commands are offered to assist these types of clients. Due to the large overhead of processing connections and XML-RPC requests, this data transfer type is not recommended and TCP/UDP should be used wherever possible.

6.2.4.1. `getMostRecentSampleValue`

Method name: `acq.getMostRecentSampleValue`

Parameters: channel index structure

Return value: double, or fault code

This procedure allows clients to read the most recent data value of a specific channel during acquisitions. In order for this call to be successful, a data acquisition must be in progress and the channel must be enabled for most recent sample value data delivery. Issue a `changeMostRecentSampleValueDeliveryEnabled` call prior to acquisition to allow this procedure to be used.

6.2.4.2. `getMostRecentSampleValueArray`

Method name: `acq.getMostRecentSampleValueArray`

Parameters: none

Return value: array of structures with channel info and values, or fault code

This procedure allows clients to retrieve the most recent data values of all channels during acquisitions in a single call. If clients are interested in the values of multiple channels, using this method is more efficient than performing consecutive `getMostRecentSampleValue` calls (which require an individual POST request per call).

Only channels that are enabled for most recent sample value data delivery will be returned. Issue a `changeMostRecentSampleValueDeliveryEnabled` call for each desired channel prior to acquisition in order for this to return the value for a channel.

The return value is an array of structures, one per channel. Each structure contains two members. The "channel" member will contain a channel index structure with members set to appropriate type and index information for the channel. The "value" member will be a double that contains the most recent sample value of the channel specified in the "channel" member.

This procedure cannot be called unless there is an acquisition in progress and there is at least one channel enabled for most recent sample value delivery.

6.2.5. Other Control Connection Commands

Control connections also allow for the following additional commands to be used by clients:

6.2.5.1. loadTemplate

Method name: `acq.loadTemplate`

Parameters: base64 encoded binary AcqKnowledge graph template

Return value: 0 on success, or fault code

Attempts to open the passed template within the AcqKnowledge environment. The parameter is a base 64 encoded AcqKnowledge "gtl" graph template file. Only templates in PC AcqKnowledge 3.7.1 or Mac AcqKnowledge graph templates are allowed to be used with this command. The parameter must include the entire contents of the template file.

The parameter in the XML-RPC call should be a base64 parameter with the raw binary contents of a graph template file. When this parameter is decoded, it should correctly provide the contents of a template file on disk.

This function will return 0 on success, otherwise a fault code if the template could not be loaded. Once a template is loaded, the hardware settings contained within that template will be used for subsequent data acquisitions. Templates for ACQ 3.9.2 and higher will retain any data connection settings that have been specified by clients. After loading a template, clients should re-send any configuration information for ports and data connection methods if they do not match the new settings from the template.

If the template data is corrupted or is incompatible with AcqKnowledge, user interaction may be required on the server computer to dismiss any error messages when attempting to load the template. If user interaction is required on errors, this call may not return until the user interaction has completed. Clients who are using XML-RPC bindings that offer timeout services may wish to use them with this function.

6.2.5.2. getAcquisitionInProgress

Method name: `acq.getAcquisitionInProgress`

Parameters: none

Return value: boolean

Query whether data acquisition is currently in progress or not. A value of true is returned if data acquisition is occurring in any open AcqKnowledge graph window.

6.2.5.3. toggleAcquisition

Method name: `acq.toggleAcquisition`

Parameters: none

Return value: 0 on success, else fault code

Toggles data acquisition in the frontmost graph. If data acquisition is in progress, it is halted. If none is in progress, data acquisition is started in the graph.

Note that this function invocation may block if physical user interaction is required to start the acquisition in the graph, such as dismissing an overwrite warning, warnings on incompatibilities between different MP unit types, specifying a save location for ac-

quisition to disk, etc. If the implementation of the XML-RPC binding used by the client supports timeout capabilities, it is highly recommended to enable timeouts for this function.

6.2.5.4. setOutputChannel

Method name: `acq.setOutputChannel`

Parameters: channel index structure, float output value

Return value: 0 on success, else fault code

Changes the voltage on the specified output channel of the MP device. For analog outputs, the value should be in the range (-10, 10) for the voltage level. For digital outputs, a value of 0 will turn the specified line off, a non-zero value will turn it on. The latency of when the output line will be changed is variable and non-guaranteed. Additionally, the output channel may be modified by other areas of the software including control channels, manual user intervention, configured stimulators, etc.

The valid output channels are dependent on the type of MP device that is connected. MP100/MP150 units allow analog 0, 1 and all digital output channels; MP35/MP36 units allow no analogs and digital 0-8.

6.2.5.5. insertGlobalEvent

Method name: `acq.insertGlobalEvent`

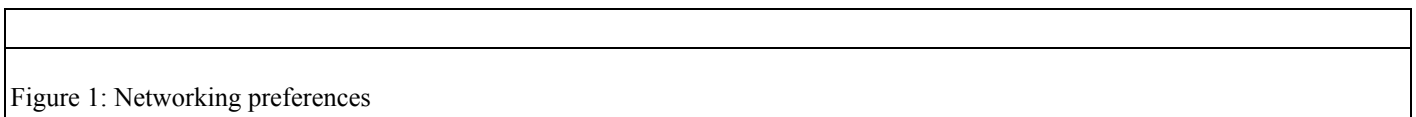
Parameters: event label string, event type string, msec since epoch as string

Return value: 0 on success, else fault code

Inserts an event into the graph, functional only when an acquisition is in progress. This inserts global events only. All three parameters of the command are formatted as strings. The event label string first parameter should be set to any text to be associated with the event. The event type string should be a four character long event type code. If no specific type is desired, use "defl" for the default event type. The final parameter is the milliseconds since the start of the epoch formatted as a string, base 10 ASCII. The timestamp is sent as a string in order to provide millisecond level accuracy and should be the number of milliseconds since midnight, Jan 1 1970, Coordinated Universal Time. The marker will be added into the graph at that time relative to the clock time when the acquisition was started. If the timestamp parameter is empty the marker will be inserted at the time when the XMLRPC request is processed.

AcqKnowledge Network Preferences

Most of the settings for the network data transfer functionality are set by clients through control connections. Several settings for the AcqKnowledge server may be set directly in the AcqKnowledge preferences. Versions of AcqKnowledge that include network data transfer functionality have an additional "Network" group of settings underneath their Preferences. These Preferences are accessed from the Display menu (Vista) or from the AcqKnowledge menu (Mac OS X):



This contains the following settings that may be changed:

Networking Preference	Description
Enable networking	Allows the network data transfer feature to be turned on and off. When checked, the network data transfer feature is enabled and clients may use network data transfer to receive data and configure transfer settings. When unchecked, the network data transfer feature is disabled. AcqKnowledge will not respond to any network requests.
Control TCP port	Specifies the port on which AcqKnowledge listens for data connections from clients. Default value is 15010. This port may be changed if other applications require use of the default port. Note that only port numbers greater than or equal to 1024 should be used. If lower numbered ports are required, AcqKnowledge must be launched with special privileges (e.g. run as "root" on Mac OS X).
Respond to auto-discovery requests	When checked, AcqKnowledge will listen for UDP broadcasts on port 15012 and respond to them as documented in the "Locating AcqKnowledge Servers" section. When unchecked, Acq-

Networking Preference	Description
	Knowledge will not respond to any UDP broadcasts and clients will need to know the hostname or IP address of the computer in order to establish data connections. This may be disabled for security purposes or if other applications require the use of port 15012.